

Tomato Leaf Disease Detection

Statistical Methods in AI — Assignment 3 | Image Classification · Transfer Learning · Web Application

Arun K 2025121004 Daniel Ashish Abraham 2025121010

1. Introduction

Tomato crops are susceptible to over a dozen fungal, bacterial, and viral diseases, several of which can devastate an entire field within days. Traditional diagnosis waiting for an agronomist or consulting printed guides is too slow and inaccessible for smallholder farmers operating under time pressure.

This project delivers an end-to-end automated system: a farmer photographs a suspect leaf on their phone; the app returns a disease name, confidence score, and treatment steps in seconds, in one of five regional languages. Two pre-trained vision models run in sequence. OpenAI **CLIP** (ViT-B/32) gates the input with zero-shot validation, rejecting non-leaf images before they reach the classifier. A fine-tuned **EfficientNet-B0** then classifies the leaf across ten tomato-specific classes. The result is served via a Streamlit web application supporting English, Hindi, Tamil, Telugu, and Malayalam.

Source code is available at <https://github.com/ashdane/nineLives-SMAI-A3>.

2. Data

2.1. Dataset

The PlantVillage dataset [1], filtered to ten tomato disease classes, provides the training corpus (~18,159 images total). Two structural challenges are present: images are controlled-environment photographs with plain backgrounds (creating a domain gap with real field images), and the class distribution is imbalanced by roughly 14× between the largest class (Yellow Leaf Curl Virus, ~5,357 images) and the smallest (Mosaic Virus, ~373 images). The dataset is split 80/20 with stratified sampling.

2.2. Augmentation

To partially close the controlled-to-field domain gap, the training pipeline applies stochastic augmentations each epoch: **RandomResizedCrop**(224) at scale (0.75, 1.0), horizontal and vertical flips ($p = 0.5$, $p = 0.3$), **RandomRotation**($\pm 20^\circ$), and **ColorJitter** (brightness, contrast, saturation, hue). Validation uses only **Resize**(256) $\rightarrow$ **CenterCrop**(224) $\rightarrow$ **Normalize** so reported accuracy reflects clean inference.

3. Method

3.1. Stage 1 — CLIP Zero-Shot Gate

CLIP [2] is a dual-encoder model trained on ~400M image-text pairs. Its image encoder (ViT-B/32) and text encoder (GPT-2-style, 12 layers) both project into a shared 512-dimensional embedding space via symmetric

contrastive loss:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N} \sum_{i=1}^N \left[\log \frac{e^{\cos(v_i, t_i)/\tau}}{\sum_j e^{\cos(v_i, t_j)/\tau}} + \log \frac{e^{\cos(v_i, t_i)/\tau}}{\sum_j e^{\cos(v_j, t_i)/\tau}} \right] \quad (1)$$

At inference, cosine similarities between the image embedding and three candidate text prompts are softmax-normalised to yield a *tomato leaf probability*. No CLIP fine-tuning is performed. A three-tier threshold scheme gates the input (Table 1).

Table 1: CLIP gate thresholds

Prob.	Action	UX
< 20%	Reject	Error; re-upload prompt
20–50%	Warn	Warning + Continue / Re-upload
> 50%	Accept	Pass to classifier

3.2. Stage 2 — EfficientNet-B0 Classifier

EfficientNet [3] scales network depth d , width w , and input resolution r jointly under the constraint $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$, derived via neural architecture search. EfficientNet-B0 ($\phi = 0$) has ~5.3M parameters, making it suitable for memory- and latency-constrained deployment.

The model is initialised from ImageNet weights (**timm**) and fine-tuned in two phases:

- **Phase 1 (Epoch 1):** Backbone frozen; only the classification head trained with Adam (lr = 10^{-3} , weight decay 10^{-4}). Prevents large early-epoch gradients from corrupting pretrained representations.
- **Phase 2 (Epochs 2–3):** All layers unfrozen; lr = 10^{-4} with cosine annealing. Adapts backbone features to disease-specific textures.

Cross-entropy with label smoothing $\epsilon = 0.1$ is used, replacing the hard one-hot target with:

$$\tilde{y}_k = \begin{cases} 1 - \epsilon + \epsilon/K & k = \text{true class} \\ \epsilon/K & \text{otherwise} \end{cases} \quad (2)$$

This calibrates confidence scores so ambiguity-handling UI states activate correctly. The model reaches ~96% top-1 validation accuracy after three epochs.

3.3. System Architecture

Figure 1 shows the full pipeline; Figure 2 shows the corresponding user flow. Two ambiguity cases are handled explicitly: (i) top confidence < 40% surfaces a low-confidence notice; (ii) top two predictions within 10 pp and neither exceeding 60% surfaces both diagnoses with full detail cards.

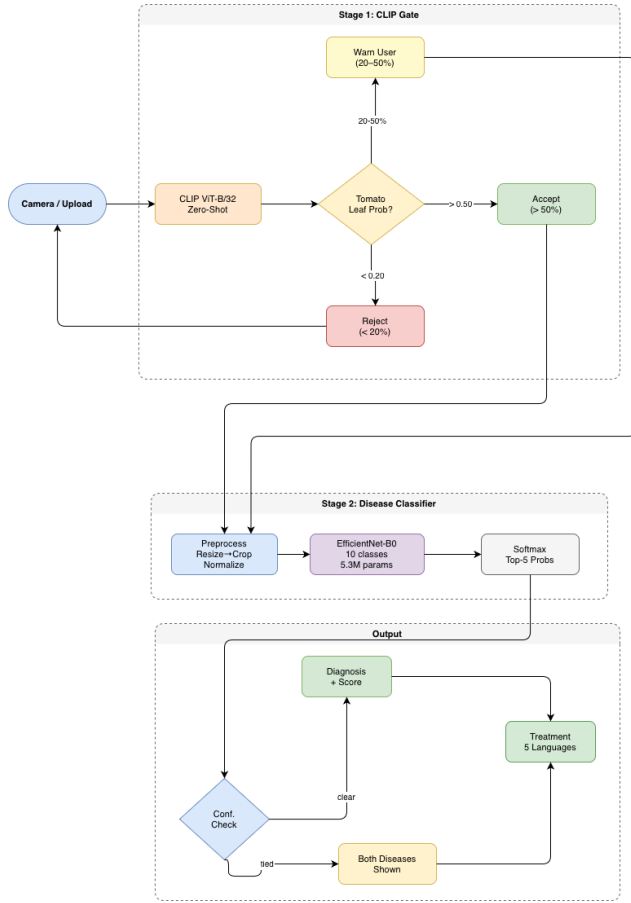


Figure 1: Two-stage pipeline: CLIP gate followed by EfficientNet-B0 classifier.

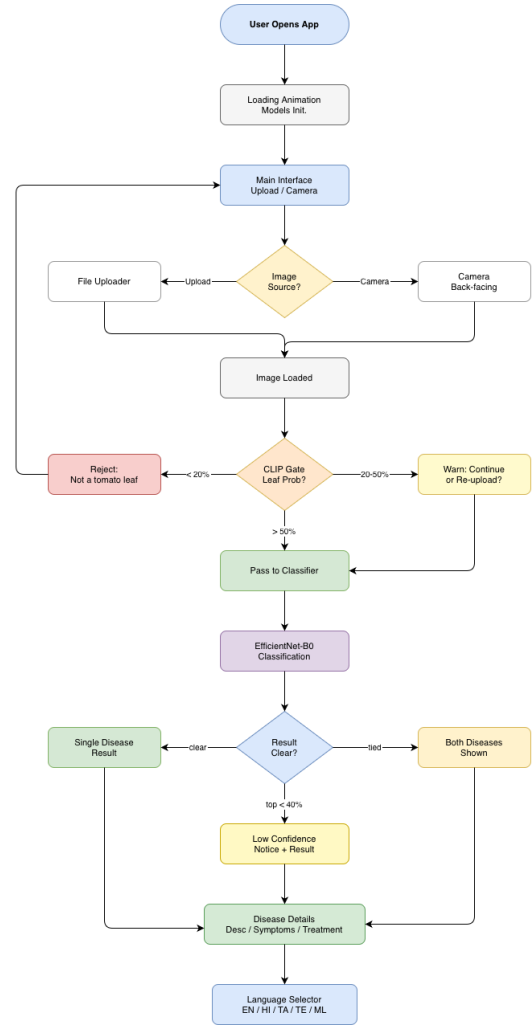


Figure 2: User flow from image input to disease diagnosis output.

4. Results

EfficientNet-B0 achieves **96% top-1 validation accuracy** on the PlantVillage tomato subset after three epochs of two-phase fine-tuning. Key behavioural outcomes observed during testing:

- **Healthy leaf:** Correctly classified at high confidence ($>90\%$) with no treatment action triggered.
- **Septoria Leaf Spot:** Detected at 54% below the 60% clear threshold correctly triggering the low-confidence UI state and top-5 breakdown.
- **Late Blight:** Detected at 79%; critical severity triggers an emergency action panel.
- **Tied prediction:** TYLCV and Late Blight both at $\sim 27\text{--}28\%$ correctly surface dual diagnosis cards.
- **CLIP rejection:** Banana leaf (0%), laptop surface (4%) hard-rejected; papaya leaf (30%) enters the warn zone with user-controlled continuation.

Multi-language output (EN / HI / TA / TE / ML) switches instantaneously from pre-generated JSON translations, with no network dependency.

5. Ablation

Table 2 summarises the impact of each major design decision.

Table 2: Ablation results

Configuration	Metric	Effect
No CLIP gate	Input safety	Notebook page classified as TYLCV at 68%; any image returns a confident (wrong) diagnosis
Binary CLIP gate (50% hard)	UX	Blurry/non-standard leaves rejected with no recourse
Three-tier gate (20–50% warn)	UX + safety	Borderline inputs flagged; user retains agency
Phase 1 only	Val. accuracy	$\approx 89\%$; backbone does not adapt to disease textures
Phase 2 only	Stability	Loss spikes; pretrained features corrupted in early epochs
Two-phase (P1+P2)	Val. accuracy	$\approx 96\%$; stable training curve
No label smoothing	Calibration	Near-100% confidence on many outputs; ambiguity states rarely trigger
Label smoothing ($\epsilon = 0.1$)	Calibration	Tied/low-confidence states activate correctly
Runtime translation API	Latency	1–3 s delay; fails under poor connectivity
Pre-generated JSON	Latency	Instant; fully offline

Three findings stand out. The CLIP gate is **essential for safety**: without it, the closed-set classifier confidently hallucinates disease labels for arbitrary inputs. Label smoothing is **necessary for UX**: without it, the tied-prediction warning almost never fires. The two-phase training contributes a **meaningful 7 pp accuracy delta** over head-only fine-tuning.

6. Limitations

Domain gap. Training exclusively on controlled-environment PlantVillage images means real-field accuracy is expected to be lower than the reported 96%; a field validation study is needed.

Class imbalance. The $14\times$ skew between Yellow Leaf Curl Virus and Mosaic Virus is unaddressed; weighted sampling or focal loss would improve minority-class recall.

Threshold sensitivity. The 20%/50% CLIP thresholds were set empirically rather than via a principled precision–recall search on a held-out non-leaf set.

Single-leaf assumption. Images with overlapping leaves, stems, or fruit are not handled; the classifier produces a prediction regardless.

No severity staging. Early vs. advanced disease stages differ visually and in treatment urgency; the current system classifies disease type only.

7. References

References

- [1] Hughes, D.P. & Salathé, M. (2015). *An open access repository of images on plant health*. arXiv:1511.08060.
- [2] Radford, A. et al. (2021). *Learning transferable visual models from natural language supervision*. ICML 2021.
- [3] Tan, M. & Le, Q. V. (2019). *EfficientNet: Rethinking model scaling for CNNs*. ICML 2019.
- [4] Wightman, R. (2019). *PyTorch Image Models*. <https://github.com/rwightman/pytorch-image-models>.
- [5] Szegedy, C. et al. (2016). *Rethinking the Inception architecture*. CVPR 2016.